

Color Manifesto

A structural model for color systems in digital products

Color is not a palette.

Color is not a token.

Color is a decision made in context.

1. The real problem with modern color systems

In small digital products, color is rarely a structural problem. A simple palette, a few manual variations, and some common sense are usually enough. The problem appears when the product grows — not only in the number of screens, but in complexity, states, contexts, and the number of people involved.

That is when color systems begin to crack. Small exceptions accumulate. Adjustments made “just for this case” start repeating. What used to be a clear palette turns into a diffuse set of historical decisions, hard to justify and even harder to maintain.

This collapse rarely happens due to lack of talent. Designers know how to choose colors. They know how to create contrast. They know how to organize visual hierarchy. What is missing is not aesthetic sensitivity, but **conceptual structure** capable of sustaining those decisions over time.

When a system does not provide a clear way to express intention, hierarchy, and context, these decisions end up being encoded implicitly — in names, variations, exceptions, and undocumented conventions. The system keeps working, but it stops being understandable.

The outcome is familiar: colors that “work here but not there,” components that require specific adjustments depending on the background, endless discussions about “which shade to use,” and a constant sense that visual consistency depends more on the team’s memory than on the system itself.

Palette Kit was born from the realization that this problem is not local or merely aesthetic. It is **structural**. And like any structural problem, it cannot be solved by simply adding more rules or more tokens.

2. When “tokens” become a silent antipattern

Color tokens emerged as a legitimate solution to a real problem: centralizing values and enabling technical consistency. In that role, they work well. The problem begins when tokens are used as a **substitute for thinking**.

As a product grows, new tokens are created to capture increasingly specific nuances: `primaryHover`, `dangerTextMuted`, `secondaryBackgroundActive`. Each name attempts to describe a particular case. Each new token solves a local problem — and creates a global one.

The antipattern takes hold when a single token carries multiple dimensions at once: where the color appears, what it means, which state it represents, and how intense it should be. These dimensions collapse into a name that sounds descriptive, but is conceptually opaque.

The side effect is combinatorial explosion. For every new context variation, a new token is created. For every exception, yet another name. The system grows in volume, but loses legibility. People no longer

know *why* a color exists — only *where* it is used.

The problem is not tokens themselves. The problem is **asking tokens to solve a conceptual problem**. Tokens store decisions; they do not explain decisions. When the underlying mental model is fragile, tokens only crystallize that fragility.

Palette Kit does not reject tokens as a technical tool. It rejects the idea that tokens can be the **primary model** of a color system. Before storing values, you need a structure for decisions.

3. Color is not a value — it is a relationship

A large portion of the fragility of color systems comes from an implicit assumption: that colors are absolute values that can be chosen, stored, and reused independently of context.

In human perception, this is simply not true.

A color is not light or dark by itself. It is light or dark **in relation** to something else. A tone can feel vibrant in one context and dull in another. Contrast that works on one screen can fail completely on another, even if the numeric values did not change.

This relativity is not a technical detail. It is the foundation of visual perception. The human eye interprets colors by comparison with what surrounds them. Ignoring this creates systems that only look consistent while the context stays static.

When a system treats color as an absolute value, it pushes adaptation work onto the designer, who ends up manually correcting every new combination of background, surface, and state. The system stops being support and becomes friction.

Recognizing that color is relational changes the question. Instead of “which color is this element?”, the more meaningful question becomes “how does this element relate visually to what surrounds it?”. That shift prepares the ground for a more robust system — but it demands abandoning simplistic models.

4. The structural mistake: mixing usage, meaning, and state

The moment many systems truly break is when they try to answer different questions using the same mechanism. Usage, meaning, state, intensity, and contrast are treated as variations of one single axis. The result is a confused vocabulary and decisions that are hard to isolate.

When a token called `primaryHoverText` fails, the problem is not only the value. From the name alone, you cannot tell **which decision is wrong**. Is it intention? State? Contrast? Context? Everything is mixed together.

This coupling makes the system brittle. Changing one dimension affects all others. Small changes generate unpredictable side effects. To avoid breaking things, teams start adding exceptions — and the cycle repeats.

Palette Kit starts from a direct critique: **different conceptual dimensions require different axes**. Usage is not meaning. Meaning is not intensity. Intensity is not contrast. Contrast is not state. Each question deserves its own answer.

Separating these axes is not academic. It is the practical path to make decisions **legible, debatable, and reversible**. When something fails, you can identify exactly which dimension needs adjustment — without rewriting the entire system.

5. The central axis: where color appears

Before discussing which color to use — or even what that color means — there is a more fundamental question: **where does this color appear?**

Traditional color systems tend to start with chromatic identity (“primary”, “secondary”, “brand”), as if color existed abstractly and could be applied everywhere later. That approach assumes something false: that the perceptual function of color is independent from its spatial and semantic role in the layout.

In practice, the human eye does not perceive colors in isolation. It perceives **surfaces, symbols, contours, and layers**. The “where” is not a secondary detail; it defines the perceptual role of color from the start.

That is why Palette Kit organizes the system around a central axis called **usage** — i.e., the functional place where color appears in the interface.

This axis does not describe intention, state, or intensity. It describes the perceptual role of color in the visual field.

The minimal model assumes four foundational usages:

- **Fill:** surfaces that define space — backgrounds, cards, buttons, clickable areas.
- **Visual vocabulary:** symbols that communicate direct meaning — text, icons, emojis.
- **Lines:** structural elements — dividers, borders, grids.
- **Overlays:** transient layers — shadows, scrims, modals, focus states.

These categories are not arbitrary. They correspond to **distinct perceptual classes** with different visual expectations. A text is not perceived as a darker background. A line is not perceived as thin text. An overlay is not perceived as a solid surface. Each usage imposes its own constraints on contrast, intensity, chroma, and transparency.

Ignoring this distinction leads to a common mistake: trying to solve every color problem by tweaking values. Saturation goes up “because it didn’t stand out.” The background goes darker “because the text didn’t read.” A new token appears “because this case is special.” The system grows, but the structure doesn’t.

By making **usage** explicit, the system starts by answering the right question: *what is the perceptual role of this color at this point in the interface?* Only then does it make sense to discuss meaning, intensity, or contrast.

This axis also exposes another reality: **color perception depends on environment**.

The same color, with the same numeric values, is not perceived the same way in different contexts. A light theme does not create the same perceptual conditions as a dark theme. The environment — understood here as the set of visual conditions in which the interface is seen — fundamentally changes color reading.

This environment includes at least:

- the luminance scheme (light or dark),
- the layout’s visual context,

- and the device itself.

This is not about preference. It is about visual adaptation. The human eye adjusts its sensitivity to the surrounding luminance. A color that “works” in one environment can fail in another, even if its values are identical.

That is why “where color appears” is not only spatial; it is also **perceptual**. Fill-in-light is not perceptually the same as fill-in-dark. Text over a light surface requires a different resolution than text over a dark surface. These differences are not exceptions; they are the rule.

Once usage is explicit, the system stops treating color as absolute and starts treating color as **situated**. Color is no longer just “blue.” It becomes “blue as surface,” “blue as symbol,” “blue as layer.” Each situation demands a different resolution.

This shift allows later decisions — contrast, intensity, relationships — to be coherent, without hidden heuristics or ad-hoc manual fixes.

Instead of asking “which blue should I use here?”, the designer asks “what is the role of this color here?”. That question is harder at first, but far more stable over time.

Making **usage** the central axis establishes a clear foundation: colors are not chosen first; **perceptual roles are**. Color becomes a consequence of context, not the other way around.

From this foundation, the other axes can operate without stepping on each other. Without it, any color system eventually slides into improvisation.

Here, improvisation is replaced with structure.

6. Surfaces do not speak. Symbols do

A recurring mistake in color systems is to treat every visual application as a variation of the same thing. Text becomes “a darker background.” Icons become “text without letters.” Borders become “thin background lines.” This simplification ignores a fundamental perceptual truth: **not everything in an interface communicates the same way**.

Surfaces and symbols play radically different perceptual roles.

Surfaces — backgrounds, cards, buttons, panels — **do not speak**. They do not carry direct meaning. Their job is to create space, establish spatial hierarchy, separate areas, and suggest interaction or rest. A well-resolved surface rarely draws attention to itself. It organizes the field so other things can be read.

Symbols — text, icons, emojis — do the opposite. They **exist to be read**. Their job is to communicate meaning directly, explicitly, immediately. Text that cannot be read has failed completely. An ambiguous icon does not “organize space”; it creates cognitive noise.

Mixing these roles under the same chromatic logic is a structural error. When systems attempt to resolve text and background with the same logic, they push teams into manual compensation: fake shadows, slightly darker backgrounds “just to help,” colors that work in one place and break elsewhere.

Palette Kit formalizes this distinction via the usage axis: **fill** and **visual vocabulary** are not the same thing; they are distinct perceptual categories with distinct constraints.

Fills work primarily along **lightness** (L): layers, depth, spatial hierarchy. A lighter or darker fill does not “say something”; it positions elements relative to each other. Excess chroma on surfaces often reads

as noise, not meaning.

Visual vocabulary prioritizes **legibility and contrast**. Text and icons must separate unambiguously from what is behind them. Tolerance for ambiguity is low. A symbol that requires effort to read is not “elegant”; it is poorly resolved.

This difference explains why surface decisions often fail when applied to text. A pleasant blue as a background can be unreadable as text. An elegant gray for a card can be insufficient for an icon. This is not taste; it’s perceptual function.

A second mistake appears when teams try to compensate by tweaking values: saturate the text “to make it pop,” darken the background “to help reading.” These fixes can work locally, but they corrode global coherence.

Palette Kit avoids that cycle by assuming that **symbols always exist in relation to a surface**, and that relationship must be explicit. Text is never resolved “alone.” Icons do not have absolute colors. They are resolved *on* something.

This changes how you think about contrast. Contrast stops being a property of a single color and becomes a property of the **relationship** between symbol and surface. Light text is only light relative to a darker background. Dark icons are only dark relative to something lighter.

Separating surfaces from symbols prevents a common failure mode: forcing a single color to perform multiple perceptual roles at once. A color can be excellent as a surface and terrible as text — and that is not a system failure; it is valuable information.

This separation also reduces the number of decisions designers must make. Instead of deciding “which exact blue for this text on this specific card,” the designer makes a more meaningful choice: *this is a symbol that must be read with high contrast on this surface*.

The system resolves the rest.

When surfaces are treated as space and symbols as language, design stops being a sequence of reactive tweaks and becomes a coherent communication system. The result is not only more consistent; it is more legible, more predictable, and easier to maintain.

7. Intensity is not style. It is hierarchy

In many design systems, color intensity is treated as an aesthetic choice: a “stronger” blue because it looks modern, a “softer” green because it feels refined. This turns intensity into style — and hides its most important function: **visual hierarchy**.

To human perception, intensity is never neutral. More intense elements draw attention sooner and compete for focus. Less intense elements recede and support context. This happens regardless of the designer’s intent. It is physiology, not opinion.

When a system does not formalize this dimension, hierarchy is built through informal adjustments: “a little more saturated,” “one step up,” “this needs to stand out.” Each decision may be reasonable in isolation, but together they become inconsistent. What was an exception becomes a pattern; what was emphasis becomes noise.

Palette Kit treats intensity as an explicit axis: **level**. This axis answers a simple but fundamental question: *how present should this color be within its usage context?*

Notice what this axis **does not** do. It does not define the color’s identity, role, or contrast. It defines the **relative force** with which the color manifests in that usage.

This separation returns clarity. Designers stop using saturation to communicate meaning. A color should not be “more red” to be more dangerous or “more green” to be more correct. Meaning belongs to intent; intensity belongs to hierarchy.

This makes the process declarative: *this element needs more presence than that, this should recede, this should dominate, this should be neutral*. Those are hierarchy decisions, not taste decisions.

Intensity is also **not absolute**. A level only makes sense within a usage and an environment. A level on a surface does not behave the same as a level on a symbol. A high level in dark mode does not manifest identically in light mode. That is why level cannot be resolved in isolation; it is contextual.

This is where many systems fail. By creating universal scales (“primary-100”, “primary-500”, “primary-900”), they assume intensity transfers across contexts. In reality, this produces inconsistencies: the same step feels aggressive in one place and washed out in another.

In Palette Kit, level is not a fixed color value. It is a **relative position** in perceptual space. It signals direction and magnitude, not absolute numbers. When resolving a color, the system adapts intensity according to usage and environment, preserving the intended hierarchy.

This changes the designer’s relationship with color. The work becomes less “find the right shade” and more “define relative importance.” Designers operate where their sensitivity matters most: attention, hierarchy, reading flow.

Everything else — lightness direction, perceptual compensation, platform fit — is the system’s job. Not because designers cannot do it, but because doing it manually everywhere does not scale.

Treating intensity as hierarchy turns level into a shared language between design and implementation: explicit, predictable, communicable.

With hierarchy defined, the next question becomes unavoidable: *how do we ensure this hierarchy stays readable across contexts?*

That is where contrast stops being accidental and becomes a conscious choice.

8. Contrast is not automatic — it is a choice

In many design systems, contrast is treated as a side effect: tweak a color, check a number, pass a minimum threshold, and assume the problem is solved. This creates a dangerous illusion — that contrast can be **delegated** to the system without conscious involvement.

In reality, contrast is a decision. Always has been.

Contrast determines what can be read, what is noticed first, what disappears into the background. It shapes legibility, hierarchy, visual rhythm, and cognitive effort. Treating contrast as automatic is surrendering responsibility for a user’s actual experience.

The problem worsens when systems promise more than they can deliver: “text is always readable,” “colors are accessible by default.” These claims ignore a simple truth: **legibility depends on context** — environment, surface, lighting, device, visual acuity, and even time-on-task.

No serious system can guarantee perfect contrast in all conditions without making strong, often invisible assumptions.

Palette Kit starts from a more honest stance: **contrast is not inferred; it is declared**.

The system does not secretly decide which direction to move — lighten or darken — without explicit intent. Instead of silent heuristics, there is a conscious choice: *this element needs high contrast in this context*.

This changes the system’s role. It becomes a rigorous executor of human intent, not a complacent auto-corrector. Given context, relationship, and a contrast intent, it resolves color predictably. If the request is impossible, it fails — explicitly.

Failure here is not a bug. It is a signal that decisions are in conflict. Maybe the surface is too close in lightness. Maybe the intensity is too high. Maybe the contrast target is unrealistic for that context. The system doesn’t “make it work”; it surfaces the problem.

This avoids a common vice: silent correction. Systems that auto-fix contrast often hide important decisions. Something appears readable “by luck” until a small change elsewhere breaks it. Designers lose predictability and trust.

By requiring contrast to be explicit, the system returns clarity. Designers do not need to compute numbers, but they must declare intent: *this text must be comfortably readable, this icon can be subtle, this label must pop*. These are legitimate design decisions that should not be implicit.

Contrast is not binary. There is comfortable reading, possible reading, quick scanning, prolonged reading. Systems that operate only on minimum thresholds ignore that perceptual spectrum.

Palette Kit treats contrast not as a magic threshold but as a **target** in perceptual space, to be pursued consistently given usage, relationship, and environment. When it cannot be reached, the impossibility is communicated.

This also reframes WCAG and APCA: important tools, but not oracles. They describe limits and probabilities, not universal guarantees. Using them without understanding their assumptions is as bad as ignoring them.

Contrast is where intention meets perception. Automating it fully loses nuance. Ignoring it loses legibility. Declaring it consciously is the mature middle path.

With this foundation, perceptual color space stops being a technical detail and becomes the system’s bedrock.

9. OKLCH: why perceptual color space changes everything

For a color system to be predictable, intentions are not enough. **Numeric changes must map to perceived changes**. This is where many systems fail: they operate in convenient math spaces, but perceptually inconsistent ones.

Spaces like RGB or HSL were built to represent colors on devices, not to model perception. In RGB, distance between numbers does not correspond to perceived difference. Tiny numeric changes can look huge in one region and barely noticeable in another.

When systems try to build hierarchy and contrast on such spaces, they inevitably fall back to empirical tuning: eyeballing, exceptions, and extra tokens. The issue is not execution; it is the foundation.

A **perceptual space** aims for a different principle: equal mathematical distances should correspond, roughly, to equal perceived differences for an average observer. Not perfect, but far closer to how we see.

OKLCH is relevant here.

OKLCH is a cylindrical representation of OKLab. It organizes color into three conceptually clear components:

- **H (Hue)**: chromatic identity — what we casually call the “color” (blue, green, red).
- **L (Lightness)**: perceived lightness, from dark to light.
- **C (Chroma)**: perceived intensity — how “vivid” or “muted” the color feels.

This is not just elegant; it is functional. Each axis corresponds to a distinct perceptual dimension. Move along L to change lightness without changing identity. Adjust C to change presence without changing lightness. Rotate H to change identity without necessarily changing intensity.

In other words: the questions designers ask about color finally have their own axes.

Contrast resolution mostly lives in L. Hierarchical intensity mostly lives in C. Meaning largely lives in H. These are not perfectly independent, but they are no longer arbitrarily entangled.

This decoupling is what allows Palette Kit to be deterministic. When the system must “go lighter” or “go darker” to hit contrast, it knows exactly which direction to move. No trial-and-error. No unpredictable compensations.

Designers do not have to choose these directions. They choose intent, level, and contrast needs. Usage and relationship provide the remaining information so the system can execute consistently.

OKLCH is not the final delivery format. It is the **internal resolution space**. The system resolves in OKLCH for perceptual coherence, then serializes to platform formats (sRGB, Display-P3, hex, rgba) at the end.

That distinction matters: designers don’t need to “think in OKLCH.” They need a system that behaves predictably under the hood.

Without a perceptual space, “increase contrast” and “reduce intensity” are vague. With OKLCH, they become clear, reproducible operations.

This sets up the next layer: contrast metrics and accessibility, honestly understood.

10. APCA vs WCAG: legibility is not a fraction

For years, accessibility in UI has been treated as a compliance problem. Measure a number, compare to a minimum, and move on. This brought important progress, but it also fossilized a dangerous misconception: that **legibility can be reduced to a single ratio**.

WCAG popularized contrast as a fixed luminance ratio. It was historically useful — simple, auditable, widely applicable. But normative simplicity is not perceptual fidelity.

The human eye does not perceive contrast as a symmetric ratio. Perception is **asymmetric** and depends on polarity, text size, weight, and context. Light text on dark does not feel the same as dark text on light, even with the same numeric ratio.

This is where traditional metrics fail quietly. They “pass” while producing uncomfortable reading, fatigue, or ambiguity. Designers feel this, but systems lack the vocabulary to describe it.

APCA (Advanced Perceptual Contrast Algorithm) attempts to model contrast closer to perception: polarity, non-linear response, and real reading conditions. The result is not simply a “better number,” but a more honest one — a probabilistic signal, not a universal stamp.

In Palette Kit, this distinction matters. Metrics are not approval badges; they are **verification instruments**. APCA is primary because it aligns better with perceptual resolution. WCAG remains as fallback and compatibility reference, not as absolute truth.

This is not ideology; it is pragmatism. Teams still must communicate with existing standards. Ignoring WCAG is irresponsible. Treating it as the only perceptual truth is equally irresponsible.

No metric replaces judgment. A contrast that passes WCAG can still be unpleasant in prolonged reading. A contrast that narrowly fails can still be acceptable in context. Legibility is not binary; it is a spectrum.

Palette Kit does not promise “everything is accessible.” It helps you aim, measure, and resolve consciously — and it makes conflicts visible when they exist.

Now that contrast is a declared intent and perceptual space is established, we can talk about the central operational truth: colors must be resolved **relative to each other**.

11. To resolve colors is to resolve relationships

One of the most persistent illusions in interface design is that colors can be chosen in isolation. Define text, define background, define border — as if each decision lived in its own compartment. It might look fine in one scenario, but it rarely survives context changes.

In perception, **colors never exist alone**. They are always seen relative to surrounding colors. Text is not light or dark “by itself.” Shadows are not subtle or intense in absolute terms. These are relational judgments.

Ignoring this relational nature is what makes systems brittle. Isolated tokens only work while the context remains fixed. Change the surface, the theme, reuse the component — and everything must be recalibrated.

Palette Kit starts from a different premise: **colors are resolved from relationships, not absolute values**.

This is captured by the relational axis: instead of only “what color is this?”, the system asks “what is this color’s relationship to another one?”. The foundational relationships are simple but sufficient: *on*, *over*, and *under*.

- **on**: direct on-surface relationships — text on background, icon on button. Contrast and readability dominate.
- **over**: partial overlays — shadows, scrim, focus layers. Depth and separation dominate more than direct readability.
- **under**: underlays — subtle glows, projected depth cues. Structural influence without becoming a focal element.

These are not arbitrary states. They are distinct perceptual interactions. Treating text like a shadow or a shadow like text produces incoherent results.

Making relationships explicit removes guesswork: lighten or darken? Increase presence or reduce it? In non-relational systems these are fragile heuristics. In relational systems the direction comes from context.

When text needs higher contrast on a light surface, the direction is toward lower lightness. On a dark surface, the direction reverses. This is not aesthetic preference; it is a consequence of the *on* relationship.

Designers do not need to decide these directions. They decide intent, level, and contrast needs. Relationship provides the missing context so the system can execute consistently.

This enables composability: components can be reused across surfaces and themes because their colors are resolved relationally, not hard-coded.

At this point, the system stops being a palette and becomes a **model**.

And once you build a model, you must decide what it refuses to do — because silent “helpfulness” is where trust dies.

12. A deterministic system, without magic

As systems grow, there is a recurring temptation: make them “smarter.” Guess intent, silently fix choices, adjust values “to help.” In the short term it feels convenient. In the long term it destroys trust.

Magic is comfortable while it works. When it fails, nobody knows why.

Palette Kit rejects this approach. It does not try to be clever. It tries to be **predictable**.

Determinism here does not mean rigidity. It means that given the same declared decisions, the result is always the same. No hidden heuristics. No invisible corrections. No “special case” behavior.

This choice has a cost: the system must say “no” more often.

When a combination cannot be resolved perceptually — for example, demanding high contrast on a surface whose chosen intensity makes it impossible — the system does not invent an intermediate hack. It fails. And the failure is intentional.

In magical systems, failures are hidden. The system “makes it work” by silently changing other axes. That creates conceptual debt: invisible decisions that permanently shape the design but were never consciously made.

In Palette Kit, failure is communication.

A failure means decisions conflict. Maybe intensity is too high. Maybe the surface is too close in lightness. Maybe the contrast goal is unrealistic. Instead of masking the problem, the system exposes it.

This changes the designer’s relationship with the system. The system stops being a complacent assistant and becomes a rigorous interlocutor. It doesn’t replace judgment; it challenges it.

Determinism also eliminates historical dependence. In magical systems, outcomes depend on accumulated exceptions. The system develops an implicit memory. Palette Kit does not. Each resolution is computed from explicit inputs, each time.

This makes the system stable over time and makes collaboration easier: if something breaks, you can locate *which axis* caused it, rather than arguing about “the color.”

Determinism removes unnecessary ambiguity — not creativity.

And that brings us to the most explicit part of the manifesto: what the system refuses to do.

13. What this system deliberately refuses to do

Good systems are defined as much by what they refuse as by what they enable. In color systems, this refusal is essential because “auto-fix everything” is always tempting.

Palette Kit refuses to **guess intent**. If intent is not declared, it does not exist. No inference from naming, no “smart defaults” pretending to know what the designer meant.

It refuses to **silently correct choices**. If a combination cannot reach the declared contrast, it does not secretly reduce intensity, flip polarity, or compensate by changing another axis. Silent correction creates invisible rules and kills trust.

It refuses to **promise automatic accessibility**. Accessibility depends on context, content, typography, and real usage. The system provides tools and targets; responsibility remains human.

It refuses to **mix conceptual axes** into a single token-like decision. No “primaryHoverTextStrong” semantics. If multiple dimensions are involved, they must be declared separately.

It refuses to treat color as an **absolute value** internally. Color is resolved relationally and contextually; “final values” exist only at serialization time.

It refuses to be **backwards-compatible with broken mental models**. It doesn’t preserve confusing conventions just because they are common.

It refuses to **optimize for undeclared edge cases**. If exceptions are frequent, it signals a modeling problem rather than absorbing them silently.

And finally: it refuses to **replace the designer**. It does not pick “beautiful colors,” does not invent hierarchy, does not decide meaning. It executes explicit decisions rigorously.

These refusals are design positions. Each one eliminates an entire class of ambiguity.

With those limits established, we can talk about the practical consequence: what changes in daily design work.

14. What changes in the designer’s daily work

After all this structure, the practical question is simple: what changes for designers in everyday work?

The short answer: **the focus shifts**.

In traditional systems, designers spend huge time on operational micro-decisions: tweak a shade because text doesn’t read, darken a surface “just a bit,” create an extra variant “only for this case.” These choices are small but expensive — and rarely reusable.

Palette Kit reframes the work. The designer’s color work concentrates on **two decisions**:

1. **Intent** — what the color means in the product’s domain
2. **Intensity (level)** — how present it should be in the hierarchy

These are real design decisions. They require product understanding and human sensitivity. They are where designers add the most value.

Everything else becomes a consequence of context: usage (where), relationship (on/over/under), environment (light/dark), and declared contrast intent.

Designers do not need to decide, repeatedly, whether text should go lighter or darker. They do not need to recalibrate every time a component is reused. Those decisions are not eliminated; they are resolved systematically based on explicit inputs.

This also changes collaboration. Conversations shift from “this blue is wrong” to “this element should have higher hierarchy” or “this information needs higher contrast.” The system translates structure into values.

Failures become diagnostic, not aesthetic. If something breaks, you don’t argue about taste; you find which axis is mis-specified.

This does not reduce expressiveness. It removes ad-hoc variation and replaces it with coherent hierarchy and meaning. Creativity moves up a level.

Less micromanagement. More real decisions.

15. Where this model can fail

No structural model should pretend to be universal. A model is trustworthy only if it admits its limits.

First: the model demands explicit decisions. Teams used to permissive systems may feel friction. No “smart defaults” that magically do the right thing for every case. Intent, level, usage, and relationships must be declared.

Second: the model does not optimize for freeform artistic exploration. If a product’s goal is to constantly break conventions, predictability may feel restrictive.

Third: the model requires adopting a more structured mental model. The long-term payoff is high, but the onboarding curve is real.

Fourth: some domains are semantically unstable. If meaning changes constantly, intent registries become moving targets.

Fifth: sophisticated perceptual spaces and conversions require good implementation. A bad implementation can erase the benefits — though the model will make failures more visible.

Finally: the model does not remove the need for visual review. Perception is too complex. The system reduces error; it does not eliminate judgment.

These are not accidental drawbacks. They are consequences of the trade: comfort now, predictability later. Implicit flexibility, explicit control. Magic, understanding.

16. Conclusion: color as system, not palette

Color stops being an empirical tweak and becomes a **teachable system**.

This manifesto does not claim final truth. It proposes a model that can be adopted, criticized, adapted, or rejected — but never hidden behind opaque naming and silent heuristics.

If it causes discomfort, questioning, or disagreement, it has done its job.